Ex. No. 9	WORKING WITH MONGODB USING SHELL COMMANDS
Date of Exercise	13/10/2026
GitHub Host Link	https://github.com/adhishm-0/web-technology-lab/tree/main/EXP9

Objective

To perform database operations such as creating a database and collections, inserting and retrieving documents, updating and deleting records, querying data, sorting results, and counting documents using MongoDB Shell commands.

Description

MongoDB is a NoSQL, document-oriented database that stores BSON documents in collections. MongoDB Shell (mongosh) provides JavaScript-based commands to select databases, create collections, insert documents, retrieve records, apply conditions, update and delete data, sort results, and count documents.

IMPORTANT COMMANDS USED :

• use databaseName — select or create a database

• db.createCollection() — create a collection

• insertOne() / insertMany() — add documents

• find() / findOne() — retrieve matching documents

• updateOne() / updateMany() with $set and $inc — modify records

• deleteOne() / deleteMany() — remove matching records

• sort(), limit(), countDocuments() — order, limit, and count results

• $gt, $lt, $gte, $lte, $in — comparison and membership query operators

Procedure

1. Start mongosh and connect to a MongoDB server.

2. Select the database required by the question using use.

3. Create the required collection and insert sample documents.

4. Run the find and findOne queries with filters and projections.

5. Modify records with updateOne() or updateMany(), then verify the changes.

6. Delete the requested records and inspect the remaining documents.

7. Sort and limit results, and use countDocuments() where required.

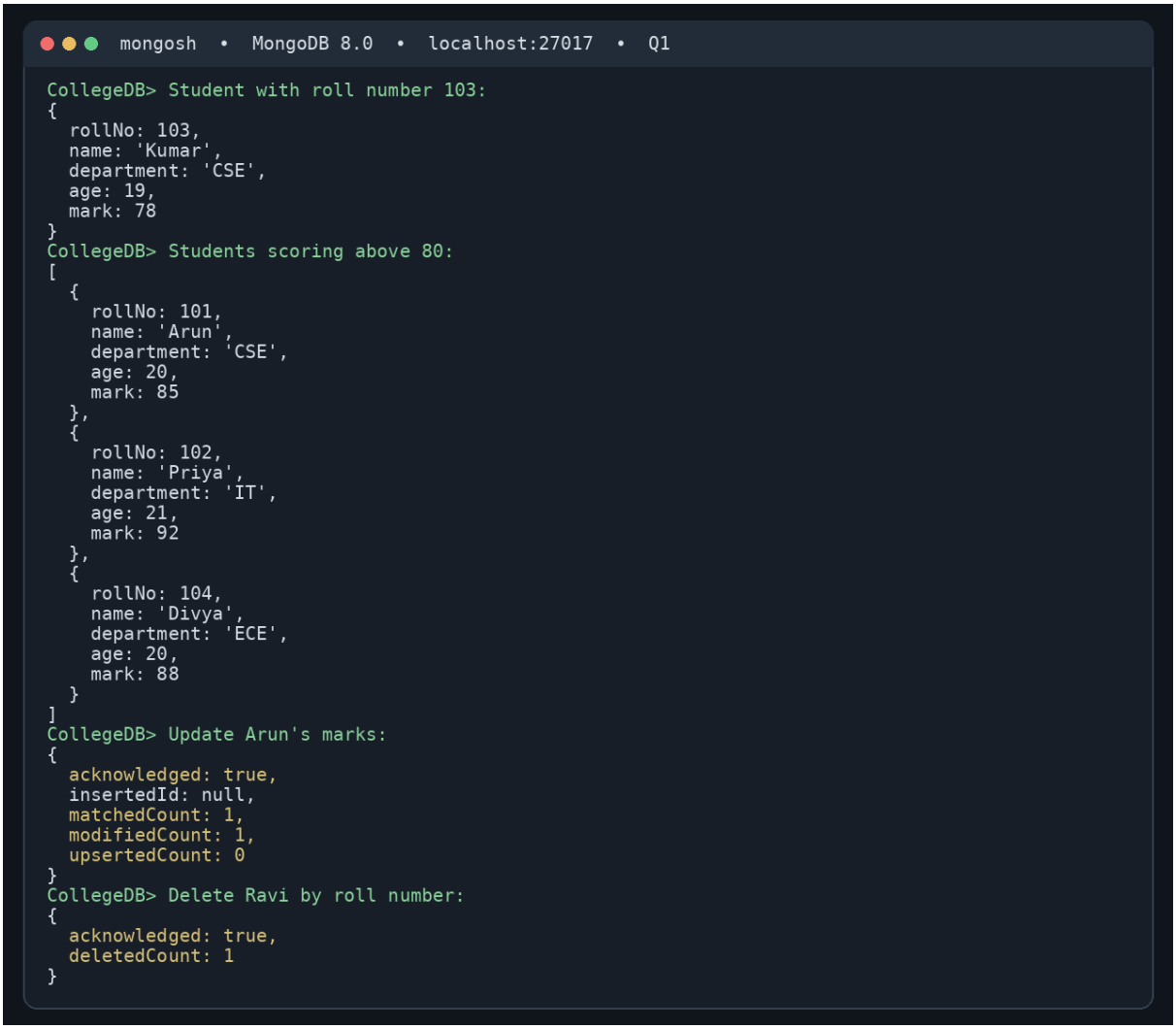
The terminal captures below show actual mongosh output from a temporary MongoDB 8.0 server. Full stdout from each run is included with the source files.

Question 1: Student Database Management

Program code:

```
// Question 1 – Student Database Management
use CollegeDB;
if (!db.getCollectionNames().includes("students")) db.createCollection("students");
const students = db.students;
students.insertOne({ rollNo: 101, name: "Arun", department: "CSE", age: 20, mark: 85 });
students.insertMany([
  { rollNo: 102, name: "Priya", department: "IT", age: 21, mark: 92 },
  { rollNo: 103, name: "Kumar", department: "CSE", age: 19, mark: 78 },
  { rollNo: 104, name: "Divya", department: "ECE", age: 20, mark: 88 },
  { rollNo: 105, name: "Ravi", department: "CSE", age: 22, mark: 67 }
]);
print("All student records:"); printjson(students.find({}, { _id: 0 }).toArray());
print("Student with roll number 103:"); printjson(students.findOne({ rollNo: 103 }, { _id: 0 }));
print("Students scoring above 80:"); printjson(students.find({ mark: { $gt: 80 } }, { _id: 0
  }).toArray());
print("Update Arun's marks:"); printjson(students.updateOne({ rollNo: 101 }, { $set: { mark: 90 }
  }));
print("Delete Ravi by roll number:"); printjson(students.deleteOne({ rollNo: 105 }));
print("Remaining students:"); printjson(students.find({}, { _id: 0 }).toArray());
```

Program Output:



Output 1: CollegeDB database and students collection: insertion, retrieval by roll number, marks query, update, and delete.

Question 2: Employee Database

Program code:

```
// Question 2 – Employee Database
use EmployeeDB;
if (!db.getCollectionNames().includes("employees")) db.createCollection("employees");
const employees = db.employees;
employees.insertMany([
  { employeeId: 201, name: "Meera", department: "HR", salary: 42000, experience: 4 },
  { employeeId: 202, name: "Rohan", department: "IT", salary: 58000, experience: 6 },
  { employeeId: 203, name: "Asha", department: "Sales", salary: 38000, experience: 3 },
  { employeeId: 204, name: "Vikram", department: "IT", salary: 72000, experience: 9 },
  { employeeId: 205, name: "Nila", department: "HR", salary: 46500, experience: 5 }
]);
print("All employees:"); printjson(employees.find({}, { _id: 0 }).toArray());
print("Employees earning more than Rs. 40,000:");
printjson(employees.find({ salary: { $gt: 40000 } }, { _id: 0 }).toArray());
print("Update employee 203 salary to Rs. 41,000:");
printjson(employees.updateOne({ employeeId: 203 }, { $set: { salary: 41000 } }));
print("Employees by descending salary:");
printjson(employees.find({}, { _id: 0 }).sort({ salary: -1 }).toArray());
print("Employees in IT:", employees.countDocuments({ department: "IT" }));
```

Program Output:

```
mongosh • MongoDB 8.0 • localhost:27017 • Q2

EmployeeDB> Update employee 203 salary to Rs. 41,000:
EmployeeDB> {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
EmployeeDB> Employees by descending salary:
EmployeeDB> [
  {
    employeeId: 204,
    name: 'Vikram',
    department: 'IT',
    salary: 72000,
    experience: 9
  },
  {
    employeeId: 202,
    name: 'Rohan',
    department: 'IT',
    salary: 58000,
    experience: 6
  },
  {
    employeeId: 205,
    name: 'Nila',
    department: 'HR',
    salary: 46500,
    experience: 5
  },
  {
    employeeId: 201,
    name: 'Meera',
    department: 'HR',
    salary: 42000,
    experience: 4
  },
  {
    employeeId: 203,
    name: 'Asha',
    department: 'Sales',
    salary: 41000,
    experience: 3
  }
]
EmployeeDB> Employees in IT: 2
```

Output 2: EmployeeDB database: salary filtering and updates, descending salary sort, and department count.

Question 3: Product Database

Program code:

```
// Question 3 – Product Database
use ProductDB;
if (!db.getCollectionNames().includes("products")) db.createCollection("products");
const products = db.products;
products.insertMany([
  { productId: 301, name: "Keyboard", category: "Accessories", price: 850, stock: 24 },
  { productId: 302, name: "Mouse", category: "Accessories", price: 550, stock: 0 },
  { productId: 303, name: "Headphones", category: "Audio", price: 1800, stock: 12 },
  { productId: 304, name: "Monitor", category: "Display", price: 12500, stock: 8 },
  { productId: 305, name: "Webcam", category: "Accessories", price: 1950, stock: 5 },
  { productId: 306, name: "Speaker", category: "Audio", price: 2300, stock: 14 }
]);
print("Accessories:"); printjson(products.find({ category: "Accessories" }, { _id: 0 }).toArray());
print("Products priced Rs. 500–2,000:");
printjson(products.find({ price: { $gte: 500, $lte: 2000 } }, { _id: 0 }).toArray());
print("Increase keyboard stock by 10:");
printjson(products.updateOne({ productId: 301 }, { $inc: { stock: 10 } }));
print("Delete out-of-stock products:"); printjson(products.deleteMany({ stock: 0 }));
print("Five most expensive products:");
printjson(products.find({}, { _id: 0 }).sort({ price: -1 }).limit(5).toArray());
```

Program Output:

```
mongosh • MongoDB 8.0 • localhost:27017 • Q3

ProductDB> Delete out-of-stock products:
{
  acknowledged: true,
  deletedCount: 1
}
ProductDB> Five most expensive products:
ProductDB> [
  {
    productId: 304,
    name: 'Monitor',
    category: 'Display',
    price: 12500,
    stock: 8
  },
  {
    productId: 306,
    name: 'Speaker',
    category: 'Audio',
    price: 2300,
    stock: 14
  },
  {
    productId: 305,
    name: 'Webcam',
    category: 'Accessories',
    price: 1950,
    stock: 5
  },
  {
    productId: 303,
    name: 'Headphones',
    category: 'Audio',
    price: 1800,
    stock: 12
  },
  {
    productId: 301,
    name: 'Keyboard',
    category: 'Accessories',
    price: 850,
    stock: 34
  }
]
```

Output 3: ProductDB database: category and price filtering, stock update, zero-stock deletion, and top-five products.

Question 4: Library Database

Program code:

```
// Question 4 – Library Database
use LibraryDB;
if (!db.getCollectionNames().includes("books")) db.createCollection("books");
const books = db.books;
books.insertMany([
  { bookId: 401, title: "Algorithms", author: "Cormen", category: "Computing", price: 950 },
  { bookId: 402, title: "Clean Code", author: "Robert Martin", category: "Computing", price: 780 },
  { bookId: 403, title: "Dune", author: "Frank Herbert", category: "Fiction", price: 620 },
  { bookId: 404, title: "Pride and Prejudice", author: "Jane Austen", category: "Classic", price:
  500 },
  { bookId: 405, title: "The Hobbit", author: "J. R. R. Tolkien", category: "Fantasy", price: 700 }
]);
print("Books by Cormen:"); printjson(books.find({ author: "Cormen" }, { _id: 0 }).toArray());
print("Computing books:"); printjson(books.find({ category: "Computing" }, { _id: 0 }).toArray());
print("Update Dune price to Rs. 680:");
printjson(books.updateOne({ bookId: 403 }, { $set: { price: 680 } }));
print("Books alphabetically by title:");
printjson(books.find({}, { _id: 0 }).sort({ title: 1 }).toArray());
print("Total books:", books.countDocuments());
```

Program Output:

```
mongosh • MongoDB 8.0 • localhost:27017 • Q4

LibraryDB> Update Dune price to Rs. 680:
LibraryDB> {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
LibraryDB> Books alphabetically by title:
LibraryDB> [
  {
    bookId: 401,
    title: 'Algorithms',
    author: 'Cormen',
    category: 'Computing',
    price: 950
  },
  {
    bookId: 402,
    title: 'Clean Code',
    author: 'Robert Martin',
    category: 'Computing',
    price: 780
  },
  {
    bookId: 403,
    title: 'Dune',
    author: 'Frank Herbert',
    category: 'Fiction',
    price: 680
  },
  {
    bookId: 404,
    title: 'Pride and Prejudice',
    author: 'Jane Austen',
    category: 'Classic',
    price: 500
  },
  {
    bookId: 405,
    title: 'The Hobbit',
    author: 'J. R. R. Tolkien',
    category: 'Fantasy',
    price: 700
  }
]
LibraryDB> Total books: 5
```

Output 4: LibraryDB database: author and category searches, price update, alphabetical sorting, and document count.

Question 5: Query Operations

Program code:

```
// Question 5 – Query Operations on students
use CollegeDB;
if (!db.getCollectionNames().includes("students")) {
  db.createCollection("students");
  db.students.insertMany([
    { rollNo: 101, name: "Arun", department: "CSE", age: 20, mark: 85 },
    { rollNo: 102, name: "Priya", department: "IT", age: 21, mark: 92 },
    { rollNo: 103, name: "Kumar", department: "CSE", age: 19, mark: 78 },
    { rollNo: 104, name: "Divya", department: "ECE", age: 20, mark: 88 },
    { rollNo: 105, name: "Ravi", department: "CSE", age: 22, mark: 67 }
  ]);
}
const students = db.students;
print("Students with marks >= 75:");
printjson(students.find({ mark: { $gte: 75 } }, { _id: 0 }).toArray());
print("Students younger than 21:");
printjson(students.find({ age: { $lt: 21 } }, { _id: 0 }).toArray());
print("Students in CSE or IT:");
printjson(students.find({ department: { $in: ["CSE", "IT"] } }, { _id: 0 }).toArray());
print("Top three students by marks:");
printjson(students.find({}, { _id: 0 }).sort({ mark: -1 }).limit(3).toArray());
print("Add 5 marks to all CSE students:");
printjson(students.updateMany({ department: "CSE" }, { $inc: { mark: 5 } }));
printjson(students.find({}, { _id: 0 }).toArray());
```

Program Output:

```
mongosh • MongoDB 8.0 • localhost:27017 • Q5

CollegeDB> Top three students by marks:
CollegeDB> [
  {
    rollNo: 102,
    name: 'Priya',
    department: 'IT',
    age: 21,
    mark: 92
  },
  {
    rollNo: 104,
    name: 'Divya',
    department: 'ECE',
    age: 20,
    mark: 88
  },
  {
    rollNo: 101,
    name: 'Arun',
    department: 'CSE',
    age: 20,
    mark: 85
  }
]
CollegeDB> Add 5 marks to all CSE students:
CollegeDB> {
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
CollegeDB> [
  {
    rollNo: 101,
    name: 'Arun',
    department: 'CSE',
    age: 20,
```

Output 5: Student queries using $gte, $lt, $in, sort(), limit(), and updateMany().

Result

The MongoDB Shell exercises were prepared to demonstrate database and collection creation, document insertion and retrieval, conditional queries, record updates and deletions, sorting, limiting, and document counting.